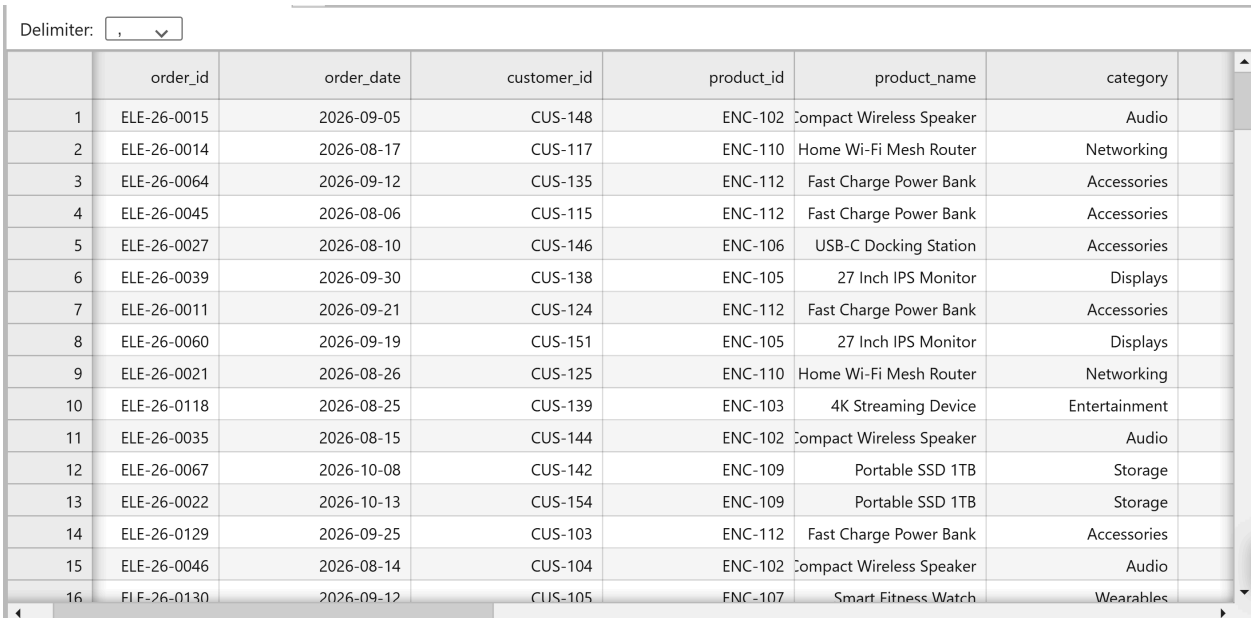
SQL Queries on E-Commerce Dataset

Cleaning the Erroneous Dataset:



Delimiter: ,

	order_id	order_date	customer_id	product_id	product_name	category
1	ELE-26-0015	2026-09-05	CUS-148	ENC-102	Compact Wireless Speaker	Audio
2	ELE-26-0014	2026-08-17	CUS-117	ENC-110	Home Wi-Fi Mesh Router	Networking
3	ELE-26-0064	2026-09-12	CUS-135	ENC-112	Fast Charge Power Bank	Accessories
4	ELE-26-0045	2026-08-06	CUS-115	ENC-112	Fast Charge Power Bank	Accessories
5	ELE-26-0027	2026-08-10	CUS-146	ENC-106	USB-C Docking Station	Accessories
6	ELE-26-0039	2026-09-30	CUS-138	ENC-105	27 Inch IPS Monitor	Displays
7	ELE-26-0011	2026-09-21	CUS-124	ENC-112	Fast Charge Power Bank	Accessories
8	ELE-26-0060	2026-09-19	CUS-151	ENC-105	27 Inch IPS Monitor	Displays
9	ELE-26-0021	2026-08-26	CUS-125	ENC-110	Home Wi-Fi Mesh Router	Networking
10	ELE-26-0118	2026-08-25	CUS-139	ENC-103	4K Streaming Device	Entertainment
11	ELE-26-0035	2026-08-15	CUS-144	ENC-102	Compact Wireless Speaker	Audio
12	ELE-26-0067	2026-10-08	CUS-142	ENC-109	Portable SSD 1TB	Storage
13	ELE-26-0022	2026-10-13	CUS-154	ENC-109	Portable SSD 1TB	Storage
14	ELE-26-0129	2026-09-25	CUS-103	ENC-112	Fast Charge Power Bank	Accessories
15	ELE-26-0046	2026-08-14	CUS-104	ENC-102	Compact Wireless Speaker	Audio
16	ELE-26-0130	2026-09-12	CUS-105	ENC-107	Smart Fitness Watch	Wearables

```
CREATE OR ALTER VIEW dbo.vw_ecommerce_clean AS
WITH exact_dedup AS (
SELECT DISTINCT * FROM dbo.ecommerce_raw
),
parsed AS (
SELECT
NULLIF(TRIM(CAST(order_id AS nvarchar(4000))), N'') AS order_id,
COALESCE(
TRY_CONVERT(date, NULLIF(TRIM(CAST(order_date AS nvarchar(4000))), N''),
23), -- yyyy-mm-dd
TRY_CONVERT(date, NULLIF(TRIM(CAST(order_date AS nvarchar(4000))), N''),
103), -- dd/mm/yyyy
TRY_CONVERT(date, NULLIF(TRIM(CAST(order_date AS nvarchar(4000))), N''),
```

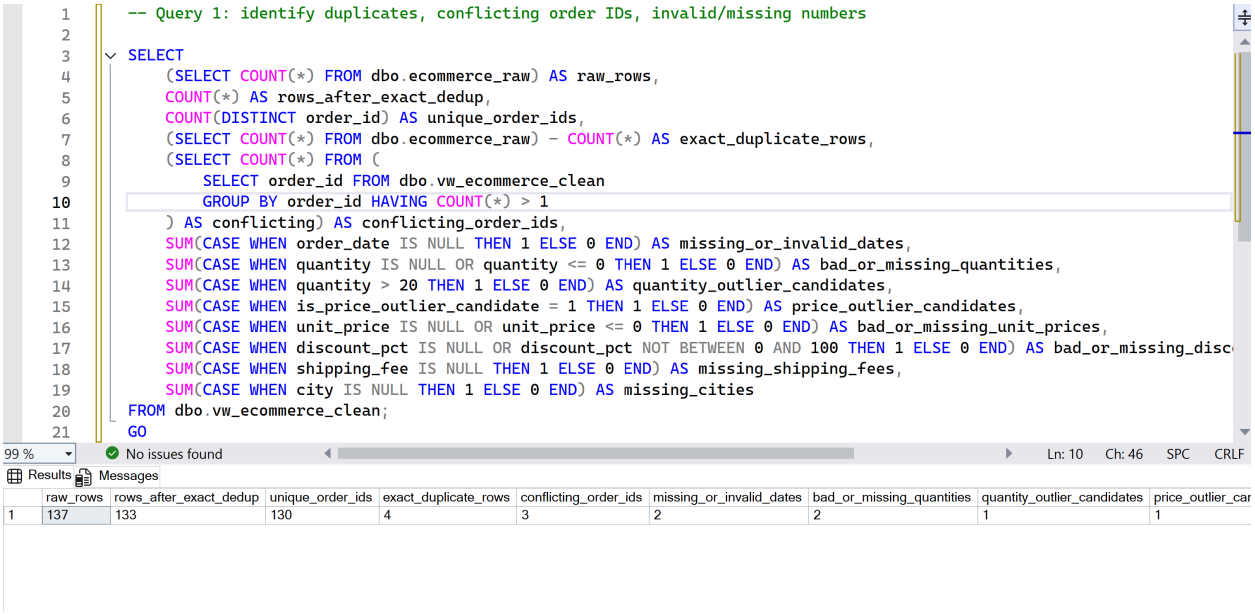
```
111) -- yyyy/mm/dd
) AS order_date,
NULLIF(TRIM(CAST(customer_id AS nvarchar(4000))), N'') AS customer_id,
NULLIF(TRIM(CAST(product_id AS nvarchar(4000))), N'') AS product_id,
NULLIF(TRIM(CAST(product_name AS nvarchar(4000))), N'') AS product_name,
CASE UPPER(TRIM(CAST(category AS nvarchar(4000))))
WHEN N'ACCESSORIES' THEN N'Accessories'
ELSE NULLIF(TRIM(CAST(category AS nvarchar(4000))), N'')
END AS category,
CASE UPPER(TRIM(CAST(city AS nvarchar(4000))))
WHEN N'BANGALORE' THEN N'Bengaluru'
WHEN N'BENGALURU' THEN N'Bengaluru'
WHEN N'CHENNAI' THEN N'Chennai'
WHEN N'COIMBATORE' THEN N'Coimbatore'
WHEN N'DELHI' THEN N'Delhi'
WHEN N'HYDERABAD' THEN N'Hyderabad'
WHEN N'MUMBAI' THEN N'Mumbai'
ELSE NULLIF(TRIM(CAST(city AS nvarchar(4000))), N'')
END AS city,
NULLIF(TRIM(CAST(state AS nvarchar(4000))), N'') AS state,
TRY_CONVERT(int, NULLIF(TRIM(CAST(quantity AS nvarchar(4000))), N'')) AS
quantity,
TRY_CONVERT(decimal(18,2),
NULLIF(REPLACE(REPLACE(REPLACE(TRIM(CAST(unit_price AS
nvarchar(4000))), N'₹', N''), N',', N''), N' ', N''), N'')) AS unit_price,
TRY_CONVERT(decimal(9,2), NULLIF(TRIM(CAST(discount_pct AS
nvarchar(4000))), N'')) AS discount_pct,
TRY_CONVERT(decimal(18,2),
NULLIF(REPLACE(REPLACE(TRIM(CAST(shipping_fee AS nvarchar(4000))), N',',
N''), N' ', N''), N'')) AS shipping_fee,
CASE UPPER(TRIM(CAST(payment_method AS nvarchar(4000))))
WHEN N'UPI' THEN N'UPI'
ELSE NULLIF(TRIM(CAST(payment_method AS nvarchar(4000))), N'')
END AS payment_method,
CASE UPPER(TRIM(CAST(order_status AS nvarchar(4000))))
WHEN N'DELIVERED' THEN N'Delivered'
```

```
WHEN N'CANCELLED' THEN N'Cancelled'
WHEN N'RETURNED' THEN N'Returned'
WHEN N'PLACED' THEN N'Placed'
WHEN N'SHIPPED' THEN N'Shipped'
ELSE NULLIF(TRIM(CAST(order_status AS nvarchar(4000))), N'')
END AS order_status,
NULLIF(TRIM(CAST(sales_channel AS nvarchar(4000))), N'') AS sales_channel,
TRY_CONVERT(int, NULLIF(TRIM(CAST(customer_rating AS nvarchar(4000))),
N'')) AS customer_rating
FROM exact_dedup
),
with_price_baselines AS (
SELECT p.*,
PERCENTILE_CONT(0.5) WITHIN GROUP (
ORDER BY CASE WHEN p.unit_price > 0 THEN p.unit_price ELSE NULL END
) OVER (PARTITION BY p.product_id) AS product_median_unit_price
FROM parsed AS p
)
SELECT p.*,
CASE WHEN p.unit_price > 5.0 * p.product_median_unit_price
THEN 1 ELSE 0 END AS is_price_outlier_candidate,
CASE WHEN p.quantity BETWEEN 1 AND 20
AND p.unit_price > 0
AND p.discount_pct BETWEEN 0 AND 100
AND p.unit_price <= 5.0 * p.product_median_unit_price
THEN 1 ELSE 0 END AS is_amount_valid,
CASE WHEN COUNT(*) OVER (PARTITION BY p.order_id) = 1
THEN 1 ELSE 0 END AS is_unambiguous_order
FROM with_price_baselines AS p;
GO


CREATE OR ALTER VIEW dbo.vw_ecommerce_approved_sales AS
SELECT
order_id, order_date, customer_id, product_id, product_name,
category, city, state, quantity, unit_price, discount_pct,
```

shipping_fee, payment_method, order_status, sales_channel, customer_rating,
CAST(ROUND(quantity * unit_price, 2) AS decimal(18,2)) AS
gross_merchandise_value,
CAST(ROUND(quantity * unit_price * (1 - discount_pct / 100.0), 2) AS
decimal(18,2)) AS net_merchandise_sales,
CAST(ROUND(quantity * unit_price * (discount_pct / 100.0), 2) AS decimal(18,2))
AS discount_amount
FROM dbo.vw_ecommerce_clean
WHERE order_status = N'Delivered'
AND is_amount_valid = 1
AND is_unambiguous_order = 1
AND order_date IS NOT NULL;
GO

Query 1: Identify duplicates, conflict datas and missing values



```
-- Query 1: identify duplicates, conflicting order IDs, invalid/missing numbers

SELECT
    (SELECT COUNT(*) FROM dbo.ecommerce_raw) AS raw_rows,
    COUNT(*) AS rows_after_exact_dedup,
    COUNT(DISTINCT order_id) AS unique_order_ids,
    (SELECT COUNT(*) FROM dbo.ecommerce_raw) - COUNT(*) AS exact_duplicate_rows,
    (SELECT COUNT(*) FROM (
        SELECT order_id FROM dbo.vw_ecommerce_clean
        GROUP BY order_id HAVING COUNT(*) > 1
    ) AS conflicting) AS conflicting_order_ids,
    SUM(CASE WHEN order_date IS NULL THEN 1 ELSE 0 END) AS missing_or_invalid_dates,
    SUM(CASE WHEN quantity IS NULL OR quantity <= 0 THEN 1 ELSE 0 END) AS bad_or_missing_quantities,
    SUM(CASE WHEN quantity > 20 THEN 1 ELSE 0 END) AS quantity_outlier_candidates,
    SUM(CASE WHEN is_price_outlier_candidate = 1 THEN 1 ELSE 0 END) AS price_outlier_candidates,
    SUM(CASE WHEN unit_price IS NULL OR unit_price <= 0 THEN 1 ELSE 0 END) AS bad_or_missing_unit_prices,
    SUM(CASE WHEN discount_pct IS NULL OR discount_pct NOT BETWEEN 0 AND 100 THEN 1 ELSE 0 END) AS bad_or_missing_disc
    SUM(CASE WHEN shipping_fee IS NULL THEN 1 ELSE 0 END) AS missing_shipping_fees,
    SUM(CASE WHEN city IS NULL THEN 1 ELSE 0 END) AS missing_cities
FROM dbo.vw_ecommerce_clean;
GO
```

99 % | No issues found | Ln: 10 | Ch: 46 | SPC | CRLF

Results | Messages

	raw_rows	rows_after_exact_dedup	unique_order_ids	exact_duplicate_rows	conflicting_order_ids	missing_or_invalid_dates	bad_or_missing_quantities	quantity_outlier_candidates	price_outlier_car
1	137	133	130	4	3	2	2	1	1

SELECT
(SELECT COUNT(*) *FROM dbo.ecommerce_raw) AS raw_rows,*
COUNT(*) AS rows_after_exact_dedup,
COUNT(DISTINCT order_id) AS unique_order_ids,
(SELECT COUNT(*) *FROM dbo.ecommerce_raw) - COUNT(*)* AS

```
exact_duplicate_rows,
(SELECT COUNT(*) FROM (
SELECT order_id FROM dbo.vw_ecommerce_clean
GROUP BY order_id HAVING COUNT(*) > 1
) AS conflicting) AS conflicting_order_ids,
SUM(CASE WHEN order_date IS NULL THEN 1 ELSE 0 END) AS
missing_or_invalid_dates,
SUM(CASE WHEN quantity IS NULL OR quantity <= 0 THEN 1 ELSE 0 END) AS
bad_or_missing_quantities,
SUM(CASE WHEN quantity > 20 THEN 1 ELSE 0 END) AS
quantity_outlier_candidates,
SUM(CASE WHEN is_price_outlier_candidate = 1 THEN 1 ELSE 0 END) AS
price_outlier_candidates,
SUM(CASE WHEN unit_price IS NULL OR unit_price <= 0 THEN 1 ELSE 0 END) AS
bad_or_missing_unit_prices,
SUM(CASE WHEN discount_pct IS NULL OR discount_pct NOT BETWEEN 0 AND
100 THEN 1 ELSE 0 END) AS bad_or_missing_discounts,
SUM(CASE WHEN shipping_fee IS NULL THEN 1 ELSE 0 END) AS
missing_shipping_fees,
SUM(CASE WHEN city IS NULL THEN 1 ELSE 0 END) AS missing_cities
FROM dbo.vw_ecommerce_clean;
GO
```

Query 2: Inspect the top 10 highest valued successfully delivered transaction

```sql
-- Query 2: Inspect the top 10 highest valued successfully delivered transaction

SELECT TOP (10)
    order_id, order_date, customer_id, product_name, city,
    quantity, gross_merchandise_value, discount_pct, net_merchandise_sales
FROM dbo.vw_ecommerce_approved_sales
WHERE net_merchandise_sales >= 15000
ORDER BY net_merchandise_sales DESC, order_id;
GO
```

90 % No issues found Ln: 9 Ch: 3 SPC C

Results Messages

	order_id	order_date	customer_id	product_name	city	quantity	gross_merchandise_value	discount_pct	net_merchandise_sales
1	ELE-26-0038	2026-09-25	CUS-138	27 Inch IPS Monitor	Hyderabad	4	71144.12	0.00	71144.12
2	ELE-26-0062	2026-09-26	CUS-128	4K Action Camera	Bengaluru	3	63117.09	10.00	56805.38
3	ELE-26-0080	2026-09-18	CUS-103	27 Inch IPS Monitor	Coimbatore	3	59445.96	5.00	56473.66
4	ELE-26-0077	2026-08-12	CUS-111	Home Wi-Fi Mesh Router	Mumbai	3	48159.06	5.00	45751.11
5	ELE-26-0067	2026-10-08	CUS-142	Portable SSD 1TB	Hyderabad	5	50092.40	15.00	42578.54
6	ELE-26-0006	2026-08-26	CUS-135	Home Wi-Fi Mesh Router	Kochi	3	47650.20	15.00	40502.67
7	ELE-26-0115	2026-08-04	CUS-108	27 Inch IPS Monitor	Coimbatore	2	39375.12	0.00	39375.12
8	ELE-26-0032	2026-08-26	CUS-109	27 Inch IPS Monitor	Coimbatore	2	39556.90	5.00	37579.06
9	ELE-26-0060	2026-09-19	CUS-151	27 Inch IPS Monitor	Coimbatore	2	36830.52	5.00	34988.99
10	ELE-26-0030	2026-10-10	CUS-136	Portable SSD 1TB	Kochi	3	31764.15	0.00	31764.15

SELECT TOP (10)
order_id, order_date, customer_id, product_name, city,
quantity, gross_merchandise_value, discount_pct, net_merchandise_sales
FROM dbo.vw_ecommerce_approved_sales
WHERE net_merchandise_sales >= 15000
ORDER BY net_merchandise_sales DESC, order_id;
GO

Query 3: Ranking cities based on successful and verified sales.

```sql
-- Query 3: Ranking cities based on successful and verified sales.

SELECT
    city,
    COUNT(*) AS delivered_orders,
    SUM(quantity) AS units_sold,
    CAST(SUM(gross_merchandise_value) AS decimal(18,2)) AS gross_sales_inr,
    CAST(SUM(net_merchandise_sales) AS decimal(18,2)) AS net_sales_inr,
    CAST(AVG(net_merchandise_sales) AS decimal(18,2)) AS avg_order_value_inr
FROM dbo.vw_ecommerce_approved_sales
WHERE city IS NOT NULL
GROUP BY city
ORDER BY net_sales_inr DESC, city;
GO
```

90 % 3 0

Results Messages

	city	delivered_orders	units_sold	gross_sales_inr	net_sales_inr	avg_order_value_inr
1	Coimbatore	11	24	275877.73	261080.66	23734.61
2	Hyderabad	7	21	174938.70	164323.44	23474.78
3	Pune	12	23	177759.97	163658.92	13638.24
4	Chennai	13	21	150223.87	131288.77	10099.14
5	Jaipur	11	23	151735.67	131185.38	11925.94
6	Kochi	5	12	124710.34	111452.65	22290.53
7	Mumbai	6	12	92480.91	89294.68	14882.45
8	Bengaluru	5	8	85729.77	76714.85	15342.97
9	Delhi	6	11	52484.03	51131.98	8522.00
10	Kolkata	5	8	55962.32	46128.68	9225.74

SELECT
city,
COUNT(*) AS delivered_orders,
SUM(quantity) AS units_sold,
CAST(SUM(gross_merchandise_value) AS decimal(18,2)) AS gross_sales_inr,
CAST(SUM(net_merchandise_sales) AS decimal(18,2)) AS net_sales_inr,
CAST(AVG(net_merchandise_sales) AS decimal(18,2)) AS avg_order_value_inr
FROM dbo.vw_ecommerce_approved_sales
WHERE city IS NOT NULL
GROUP BY city
ORDER BY net_sales_inr DESC, city;
GO

Query 4: Above Average Orders

```sql
-- Query 4: Above Average orders

SELECT TOP (10)
    a.order_id, a.category, a.product_name,
    a.net_merchandise_sales AS order_net_sales_inr,
    CAST((
        SELECT AVG(b.net_merchandise_sales)
        FROM dbo.vw_ecommerce_approved_sales AS b
        WHERE b.category = a.category
    ) AS decimal(18,2)) AS category_average_inr
FROM dbo.vw_ecommerce_approved_sales AS a
WHERE a.net_merchandise_sales > (
    SELECT AVG(b.net_merchandise_sales)
    FROM dbo.vw_ecommerce_approved_sales AS b
    WHERE b.category = a.category
)
ORDER BY a.net_merchandise_sales DESC, a.order_id;
GO
```

90 % No issues found Ln: 9 Ch

Results Messages

	order_id	category	product_name	order_net_sales_inr	category_average_inr
1	ELE-26-0038	Displays	27 Inch IPS Monitor	71144.12	41173.11
2	ELE-26-0062	Cameras	4K Action Camera	56805.38	30987.35
3	ELE-26-0080	Displays	27 Inch IPS Monitor	56473.66	41173.11
4	ELE-26-0077	Networking	Home Wi-Fi Mesh Router	45751.11	24472.69
5	ELE-26-0067	Storage	Portable SSD 1TB	42578.54	19760.64
6	ELE-26-0006	Networking	Home Wi-Fi Mesh Router	40502.67	24472.69
7	ELE-26-0030	Storage	Portable SSD 1TB	31764.15	19760.64
8	ELE-26-0078	Wearables	Smart Fitness Watch	30044.70	11440.51
9	ELE-26-0054	Networking	home wi-fi mesh router	28041.54	24472.69
10	ELE-26-0070	Networking	Home Wi-Fi Mesh Router	26983.90	24472.69

SELECT TOP (10)
a.order_id, a.category, a.product_name,
a.net_merchandise_sales AS order_net_sales_inr,
CAST((
SELECT AVG(b.net_merchandise_sales)
FROM dbo.vw_ecommerce_approved_sales AS b
WHERE b.category = a.category
) AS decimal(18,2)) AS category_average_inr
FROM dbo.vw_ecommerce_approved_sales AS a
WHERE a.net_merchandise_sales > (
SELECT AVG(b.net_merchandise_sales)
FROM dbo.vw_ecommerce_approved_sales AS b
WHERE b.category = a.category
)
ORDER BY a.net_merchandise_sales DESC, a.order_id;
GO

Query 5: Calculate Monthly Revenue

```sql
-- Query 5: Calculate Monthly Revenue

SELECT
    DATEFROMPARTS(
        YEAR(order_date),
        MONTH(order_date),
        1
    ) AS sales_month,
    COUNT(*) AS total_orders,
    SUM(quantity) AS total_units_sold,
    SUM(gross_merchandise_value) AS gross_revenue,
    SUM(discount_amount) AS total_discounts,
    SUM(net_merchandise_sales) AS net_revenue
FROM dbo.vw_ecommerce_approved_sales
GROUP BY
    DATEFROMPARTS(
        YEAR(order_date),
        MONTH(order_date),
        1
    )
ORDER BY sales_month;
```

90 % 1 0 Ln: 21 Ch:

Results Messages

	sales_month	total_orders	total_units_sold	gross_revenue	total_discounts	net_revenue
1	2026-08-01	37	74	611705.96	43117.29	568588.68
2	2026-09-01	33	63	543696.18	53979.34	489716.84
3	2026-10-01	11	26	186501.17	18546.69	167954.49

SELECT
DATEFROMPARTS(
YEAR(order_date),
MONTH(order_date),
1
) AS sales_month,
COUNT(*) AS total_orders,
SUM(quantity) AS total_units_sold,
SUM(gross_merchandise_value) AS gross_revenue,
SUM(discount_amount) AS total_discounts,
SUM(net_merchandise_sales) AS net_revenue
FROM dbo.vw_ecommerce_approved_sales
GROUP BY
DATEFROMPARTS(
YEAR(order_date),
MONTH(order_date),
1
)
ORDER BY sales_month;

Query 6: Rank Products by Revenue

```sql
-- Query 6: Rank Products by Revenue
WITH ProductRevenue AS
(
    SELECT
        product_id,
        MAX(product_name) AS product_name,
        COUNT(*) AS total_orders,
        SUM(quantity) AS total_quantity_sold,
        SUM(net_merchandise_sales) AS total_revenue
    FROM dbo.vw_ecommerce_approved_sales
    GROUP BY product_id
)

SELECT
    DENSE_RANK() OVER (
        ORDER BY total_revenue DESC
    ) AS product_rank,
    product_id,
    product_name,
    total_orders,
    total_quantity_sold,
    total_revenue
FROM ProductRevenue
```

74 % No issues found

Results Messages

	product_rank	product_id	product_name	total_orders	total_quantity_sold	total_revenue
1	1	ENC-105	27 Inch IPS Monitor	7	16	288211.74
2	2	ENC-110	Home Wi-Fi Mesh Router	10	18	244726.85
3	3	ENC-109	Portable SSD 1TB	9	20	177845.80
4	4	ENC-107	Smart Fitness Watch	11	22	125845.60
5	5	ENC-103	4K Streaming Device	9	18	93835.99
6	6	ENC-111	4K Action Camera	3	5	92962.05
7	7	ENC-106	USB-C Docking Station	6	10	71735.33
8	8	ENC-101	Noise-Canceling Headphones	5	10	41389.32
9	9	ENC-112	Fast Charge Power Bank	8	17	28947.88
10	10	ENC-102	Compact Wireless Speaker	4	9	22644.23
11	11	ENC-104	Mechanical Keyboard	3	3	20947.91

WITH ProductRevenue AS
(
SELECT
product_id,
MAX(product_name) AS product_name,
COUNT(*) AS total_orders,
SUM(quantity) AS total_quantity_sold,
SUM(net_merchandise_sales) AS total_revenue
FROM dbo.vw_ecommerce_approved_sales
GROUP BY product_id
)

SELECT
DENSE_RANK() OVER (
ORDER BY total_revenue DESC

) AS product_rank,
product_id,
product_name,
total_orders,
total_quantity_sold,
total_revenue
FROM ProductRevenue
ORDER BY product_rank;

Query 7: Compare Payment Methods by Revenue

```sql
-- Query 7: Compare Payment Methods by Revenue

WITH PaymentRevenue AS
(
    SELECT
        COALESCE(
            payment_method,
            'Not Specified'
        ) AS payment_method,
        COUNT(*) AS total_orders,
        SUM(net_merchandise_sales) AS total_revenue,
        AVG(net_merchandise_sales) AS average_order_value
    FROM dbo.vw_ecommerce_approved_sales
    GROUP BY
        COALESCE(
            payment_method,
            'Not Specified'
        )
)

SELECT
    payment_method,
    total_orders,
    total_revenue,
    ROUND(
        average_order_value, 2
```

82 % No issues found

Results Messages

	payment_method	total_orders	total_revenue	average_order_value	revenue_percentage
1	Wallet	16	319460.54	19966.280000	26.05
2	Cash on Delivery	13	261204.55	20092.660000	21.30
3	Net Banking	18	260603.80	14477.990000	21.25
4	UPI	20	227509.19	11375.460000	18.55
5	Card	13	144173.82	11090.290000	11.76
6	Not Specified	1	13308.11	13308.110000	1.09

WITH PaymentRevenue AS
(
SELECT
COALESCE(
payment_method,

```
'Not Specified'
) AS payment_method,
COUNT(*) AS total_orders,
SUM(net_merchandise_sales) AS total_revenue,
AVG(net_merchandise_sales) AS average_order_value
FROM dbo.vw_ecommerce_approved_sales
GROUP BY
COALESCE(
payment_method,
'Not Specified'
)
)

SELECT
payment_method,
total_orders,
total_revenue,
ROUND(
average_order_value, 2
) AS average_order_value,
CAST(
100.0 * total_revenue /
NULLIF(SUM(total_revenue) OVER (), 0)
AS DECIMAL(6,2)
) AS revenue_percentage
FROM PaymentRevenue
ORDER BY total_revenue DESC;
```